

ラストワンマイル 物資配送アプリ 詳細設計書

本ドキュメントは、「ラストワンマイル 物資配送アプリ」における各機能の内部処理ロジック、トランザクション境界、悲観的ロックによる整合性制御、バリデーション仕様およびエラーハンドリング機構について詳細に定義するものです。

1. 共通設計方針およびアーキテクチャ規約

1.1 トランザクション制御および排他制御方針

- トランザクション境界:

複数テーブルの更新、または参照結果に依存して更新を行う処理（在庫引当、注文取消、ステータス変更、物資削除等）は、サーブレット層にて `conn.setAutoCommit(false)` を明示的に呼び出し、同一コネクション内でコミット（`conn.commit()`）またはロールバック（`conn.rollback()`）を完結させます。

- 悲観的行ロック（`SELECT ... FOR UPDATE`）:

同時実行による在庫の過剰引当（オーバーセル）や、複数ボランティアによる同一配送依頼の重複着手を防止するため、処理対象レコードに対して `FOR UPDATE` クエリを発行して行ロックを獲得します。

- デッドロック防止策:

複数物資を同時にロック・引当する注文確定処理（`OrderConfirmServlet`）では、ロック獲得前にカート内の品目ID（`itemId`）を昇順にソート（`Comparator.comparingInt(CartItem::getItemId)`）してから順次ロックを取得することで、相互待ちによるデッドロックの発生を完全に抑止します。

1.2 セキュリティおよび整合性保護

- 二重送信・CSRF対策:

注文申請画面および物資追加画面において、`CsrfTokenUtil` により生成されたワンタイムトークンをフォームの隠しフィールド（`hidden`）に埋め込み、POST受信時にセッション値と突合・即時破棄します。

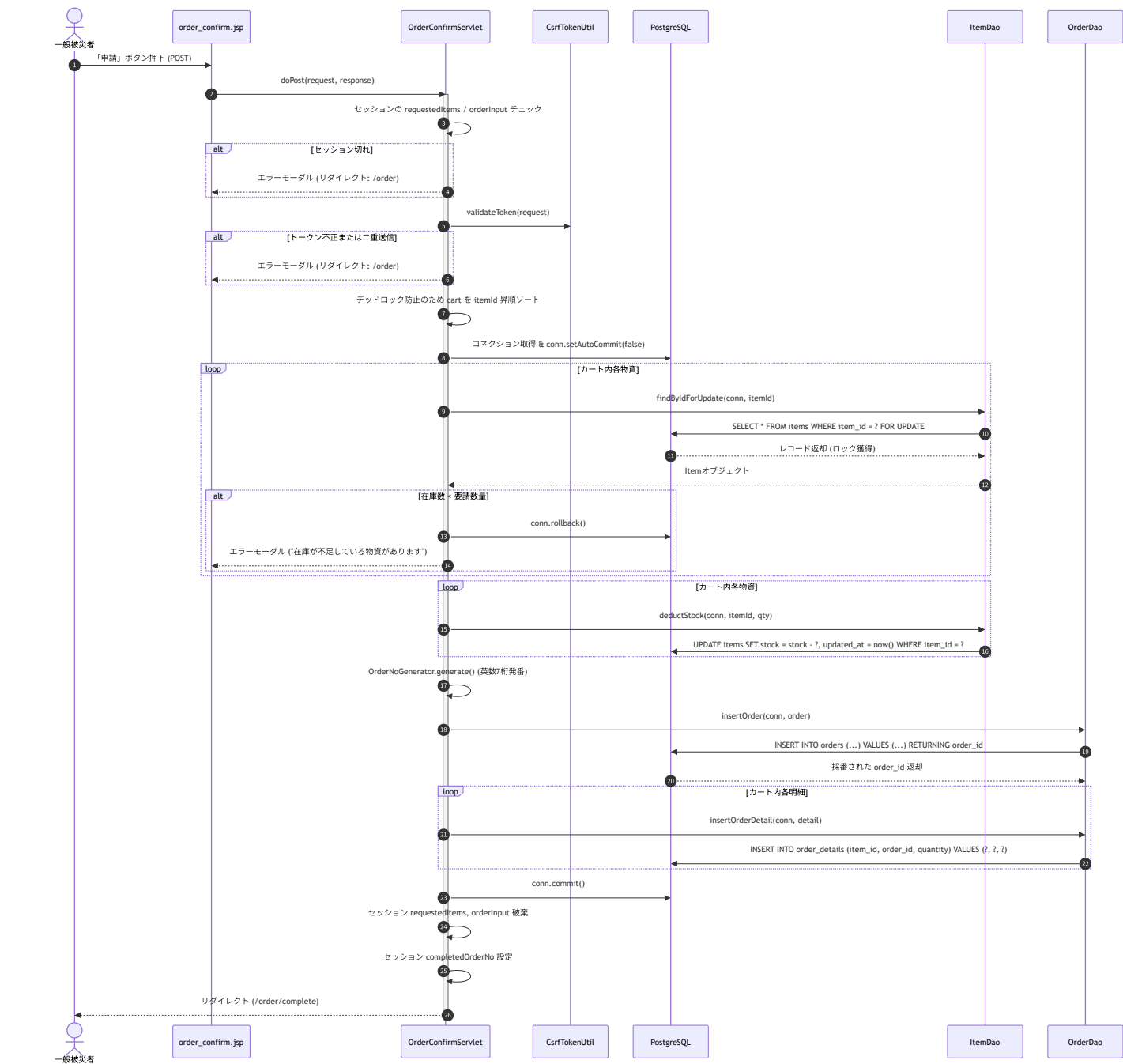
- 文字コード保証:

すべてのリクエストおよびレスポンスは `EncodingFilter` により UTF-8 に統一されます。

2. 機能別詳細処理ロジック & シーケンス

2.1 【被災者】物資申請および在庫引当処理（`OrderConfirmServlet.java`）

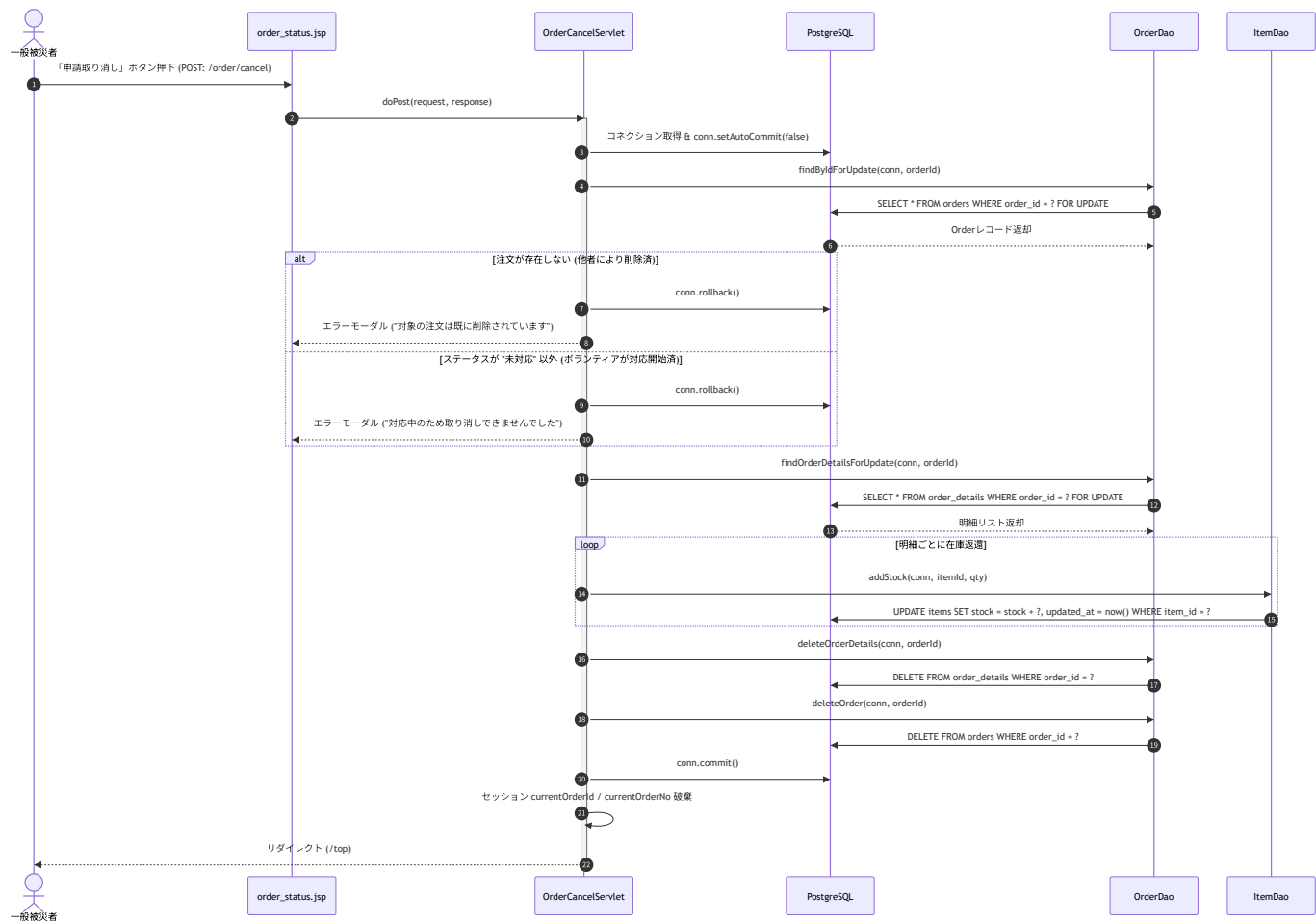
ユーザーが選択した物資を最終確認し、在庫を引き当てて注文を確定する最重要トランザクション処理です。



■ 処理アルゴリズム詳細

- 1. セッション検証:
 - session.getAttribute("requestedItems") および session.getAttribute("orderInput") が存在するか検証。未存在の場合はモーダル表示後、物資選択画面へ誘導。
- 2. トークン検証:
 - CsrfTokenUtil.validateToken(request) を実行。トークン不一致時は処理を中断。
- 3. ソートと悲観的ロック:
 - リストを itemId 昇順に並び替え後、トランザクションを開始。
 - findByIdForUpdate により対象物資の最新在庫を取得・行ロック。
 - いずれかの品目で stock < quantity の場合、即座に conn.rollback() を実行し、在庫不足エラーメッセージを出力。
- 4. 在庫減算および注文作成:
 - 全品目の在庫が十分であれば、各品目の在庫を stock - quantity に減算更新。
 - OrderNoGenerator.generate() により重複のない英数字7桁の注文番号を生成。
 - orders テーブルに注文者情報と配送ステータス "未対応" を登録し、生成された order_id を取得。
 - order_details テーブルに各品目の注文数量を登録。
- 5. コミットと後処理:
 - トランザクションをコミットし、セッション上のカート情報と住所情報を破棄。
 - 発番された注文番号を completedOrderNo にセットし、完了画面へリダイレクト。

2.2 【被災者】 注文照会および申請取り消し処理 (`OrderStatusServlet.java` , `OrderCancelServlet.java`)

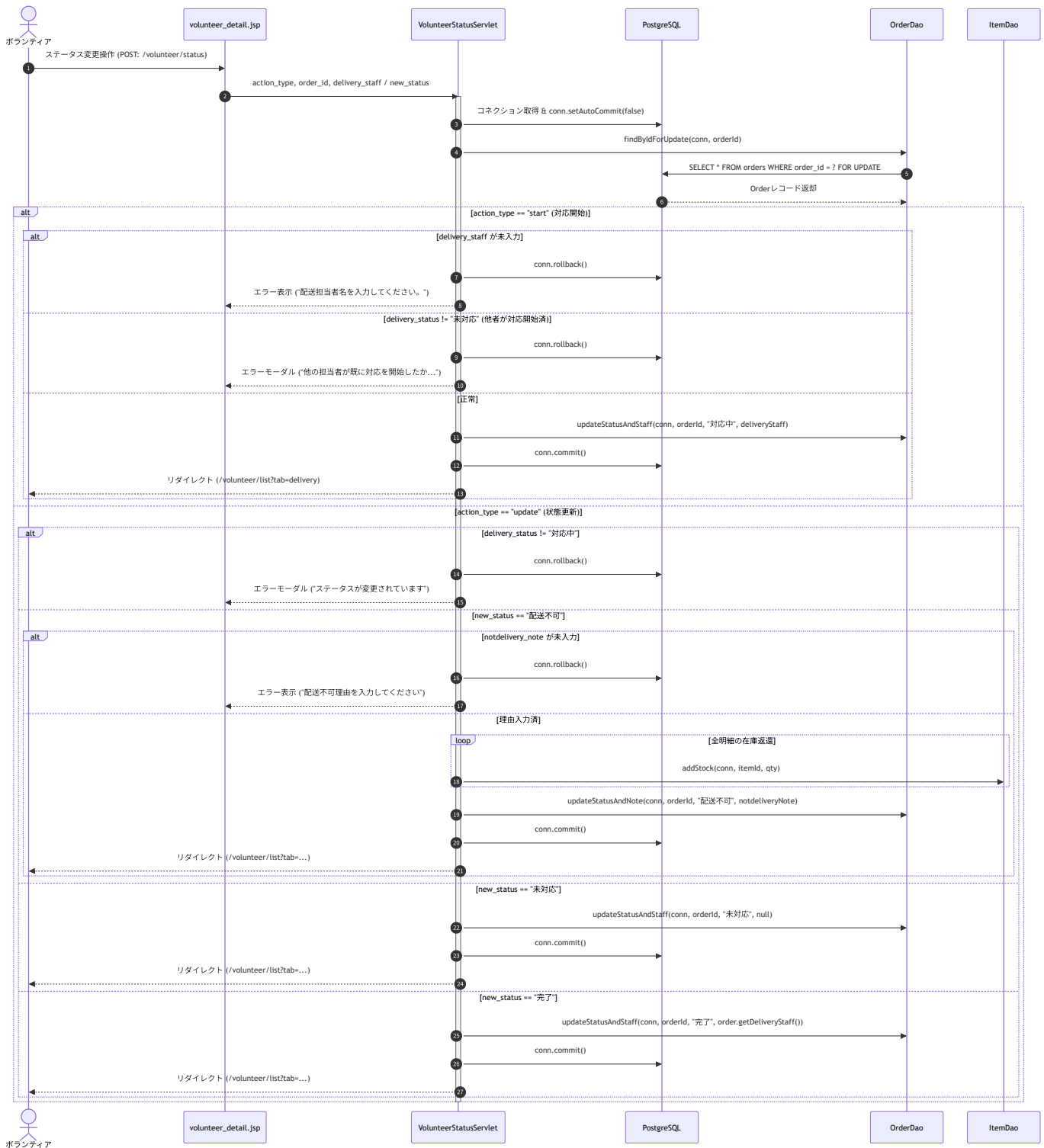


■ 処理アルゴリズム詳細

1. 取消可否の判定:
 - 注文レコードを行ロック付きで取得し、ステータスが厳密に "未対応" であることを確認。
 - 既にボランティアが対応を開始している（ステータスが "対応中" や "完了" ）場合は、ロールバックしてエラーモーダルを表示し、取消を拒絶。
2. 在庫棚戻し（返還）:
 - 当該注文に紐付く全明細（ `order_details` ）を取得し、各品目の在庫数に注文数量を加算（ `addStock` ）。
3. カスケード削除:
 - 外部キー制約の整合性を保つため、まず `order_details` のレコードを全件削除し、その後 `orders` レコードを削除。
4. コミット:
 - コミット後、トップ画面へリダイレクト。

2.3 【ボランティア】 配送ステータス更新処理 (`VolunteerStatusServlet.java`)

ボランティアによる配送業務の進捗更新ロジックです。



■ 業務ルールと自動在庫返還

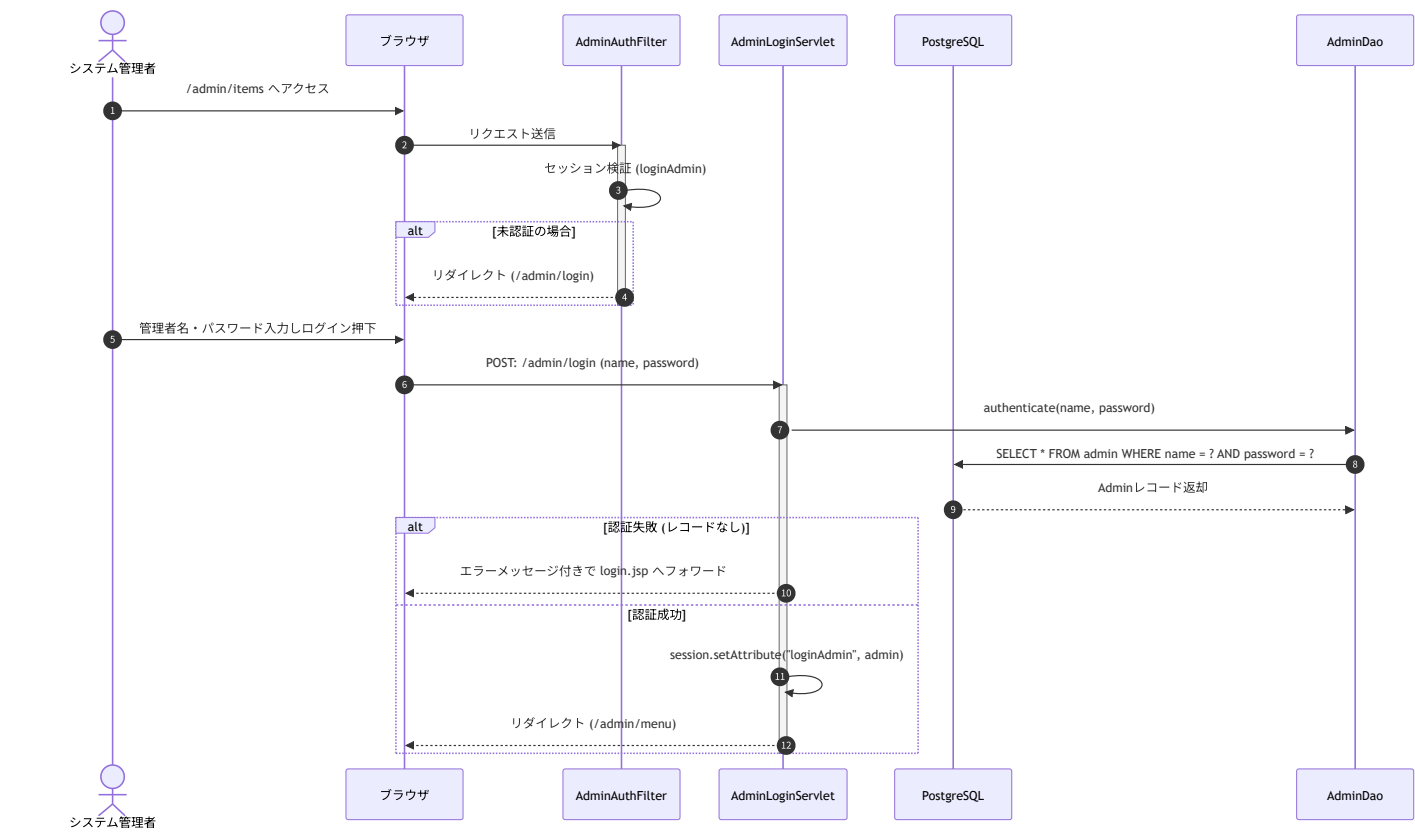
• 対応開始時の排他性:

- 同一依頼に対して複数のボランティアが同時に「対応開始」を押した場合、行ロックにより先着1名のみが "対応中" へ更新成功し、後着はエラーモーダルで検知・再読み込まれます。

• 配送不可時の在庫棚戻し:

- 道路寸断や受取人不在等により配送が不可能となった場合、物資の死蔵を防ぐため、システムが自動的に全注文明細の数量を `items.stock` に加算返還します。
- 配送不可理由は後日の検証のため `notdelivery_note` に必須入力とします。

2.4 【管理者】認証およびアクセス認可制御



2.5 【管理者】物資マスタ管理処理

(1) 新規登録 (AdminItemAddServlet.java)

- 重複検証ロジック:
 - genre_id、item_name (トリム後)、location (保管場所) の3条件が完全一致する既存レコードが存在しないかを ItemDao.isDuplicate により事前照会。
 - 重複が存在する場合は「品名と場所が重複しているため追加できません」を返し、登録を抑止。

(2) 編集 (AdminItemEditServlet.java)

- 排他更新:
 - 対象物資を行ロック (findByIdForUpdate) し、存在を確認。
 - 自身以外のレコードで同一ジャンル・品名・保管場所が存在しないかを検証 (excludeItemId を指定)。

(3) 入荷 (AdminItemStockServlet.java)

- 在庫加算:
 - 入力された入荷数量 (1以上の自然数) を stock + quantity としてアトミックに加算更新。

(4) 削除 (AdminItemDeleteServlet.java)

- 外部キー整合性チェック:
 - ItemDao.hasOrderHistory(conn, itemId) を実行。
 - クエリ: SELECT COUNT(*) FROM order_details WHERE item_id = ?
 - 過去に1件でも注文明細に利用されている物資は、DB整合性制約違反および過去履歴消失を防ぐため削除不可とし、「注文履歴が存在するため削除できません」エラーモーダルを表示。
 - 履歴が存在しない未使用物資のみ、安全に物理削除を実行。

3. 入出力バリデーション仕様一覧

画面・対象機能	入力項目	型・形式	必須	制約・バリデーション条件	エラーメッセージ
物資選択 (order_items.jsp)	qty_{item_id}	半角整数	任意	・ 0以上の整数 ・ 在庫数を超える値は自動で最大在庫数に補正 ・ 全品目の合計選択数が1以上	物資を1つ以上選
お届け先入力 (order_address.jsp)	name	文字列	必須	・ トリム後1文字以上、20文字以内	お名前を入力し お名前は20文字
お届け先入力 (order_address.jsp)	address	文字列	必須	・ トリム後1文字以上、255文字以内	お届け先住所を お届け先住所は2
お届け先入力 (order_address.jsp)	phone	半角数字	必須	・ 正規表現: ^[0-9]{1,11}\$ (11桁以内の半角数字)	電話番号を入力 電話番号は11桁
お届け先入力 (order_address.jsp)	note	文字列	任意	・ 特になし	-
状況確認 (top.jsp)	order_no	半角英数字	必須	・ 未入力またはDB未登録の注文番号	注文番号があり
ボランティア詳細 (volunteer_detail.jsp)	delivery_staff	文字列	必須 (対応開始時)	・ トリム後1文字以上	配送担当者名を
ボランティア詳細 (volunteer_detail.jsp)	notdelivery_note	文字列	必須 (配送不可時)	・ トリム後1文字以上	配送不可理由を
管理者ログイン (admin_login.jsp)	name , password	文字列	必須	・ DBの管理者テーブルと完全一致	管理者名または
物資追加・編集 (admin_item_add/edit.jsp)	genre_id	整数	必須	・ 1～5のいずれか	全項目を正しく
物資追加・編集 (admin_item_add/edit.jsp)	item_name	文字列	必須	・ トリム後1文字以上、100文字以内	全項目を正しく
物資追加・編集 (admin_item_add/edit.jsp)	stock	半角整数	必須	・ 0以上の整数	在庫数は半角数
物資追加・編集 (admin_item_add/edit.jsp)	location	文字列	必須	・ トリム後1文字以上、10文字以内	全項目を正しく
物資入荷 (admin_item_stock.jsp)	quantity	半角整数	必須	・ 1以上の整数	入荷数量は半角 入荷数量は1以上

4. 共通例外処理およびエラーハンドリング詳細

4.1 SQL例外 (SQLException) ハンドリング規約

- ・ トランザクション処理中に SQLException が発生した場合、 catch ブロック内で必ず conn.rollback() を実行します。
- ・ finally ブロックで conn.setAutoCommit(true) に復帰させた後、コネクションを確実にクローズ (conn.close()) します。
- ・ サーブレット層からは throw new ServletException(e) としてコンテナへ通知し、適切な500エラー画面または共通エラーページヘルパーティングします。

4.2 動的モータル連携仕様 (common_error_modal.jsp)

- ・ ユーザーに競合やバリデーション違反を通知する際、JSPインクルードされた common_error_modal.jsp が自動起動します。
- ・ 制御ロジック:

```
// 閉じるボタン押下時の挙動
function closeModal() {
  const action = document.getElementById('modalAction').value;
  const redirectUrl = document.getElementById('modalRedirectUrl').value;
  if (action === 'reload') {
    window.location.reload();
  } else if (action === 'redirect' && redirectUrl) {
    window.location.href = redirectUrl;
  } else {
    document.getElementById('errorModal').style.display = 'none';
  }
}
```

- これにより、データの不整合が発生した瞬間にユーザー画面を最新状態に強制同期させるUXを実現しています。